

Лабораторная работа №5

Реализация веб-приложения для обработки GET- и POST-запросов

Выполнил: Киселев Георгий Петрович
Группа: ИВТ-1.1

Цель работы

Целью лабораторной работы является разработка простого веб-приложения, которое обрабатывает GET- и POST-запросы, а также демонстрирует передачу данных между клиентом и сервером.

Используемые инструменты

Для выполнения лабораторной работы были использованы следующие инструменты:

- **Node.js** — среда выполнения JavaScript;
 - **Express.js** — фреймворк для создания веб-сервера;
 - **Visual Studio Code** — редактор кода;
 - **браузер** — для проверки GET-запросов;
 - **cURL** — для отправки GET- и POST-запросов из командной строки;
 - **Git и GitHub** — для хранения исходного кода проекта.
-

Описание веб-приложения

В рамках лабораторной работы было создано веб-приложение на Node.js с использованием фреймворка Express.

Приложение реализует следующие маршруты:

Метод	Маршрут	Описание
GET	/	Проверка работы сервера
GET	/hello?name=...	Возвращает приветствие с именем
POST	/echo	Возвращает данные, отправленные в теле запроса

Метод	Маршрут	Описание
POST	/sum	Складывает два числа, переданные в JSON

Все ответы сервера возвращаются в формате JSON.

Код веб-приложения

```
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware для чтения JSON из тела POST-запросов
app.use(express.json());
app.use(cors());

// Корневой GET-запрос
app.get('/', (req, res) => {
  res.json({
    title: 'ЛР-5. GET и POST запросы',
    message: 'Веб-приложение работает',
    routes: {
      getHello: 'GET /hello?name=Георгий',
      postEcho: 'POST /echo',
      postSum: 'POST /sum'
    }
  });
});

// GET-запрос с query-параметром
app.get('/hello', (req, res) => {
  const name = req.query.name || 'гость';

  res.json({
    method: 'GET',
    message: `Привет, ${name}!`,
    query: req.query
  });
});

// POST-запрос, который возвращает отправленные данные
app.post('/echo', (req, res) => {
  res.json({
    method: 'POST',
```

```
    message: 'Данные успешно получены',
    received: req.body
  });
});

// POST-запрос для сложения двух чисел
app.post('/sum', (req, res) => {
  const { a, b } = req.body;

  if (typeof a !== 'number' || typeof b !== 'number') {
    return res.status(400).json({
      error: 'Поля a и b должны быть числами'
    });
  }

  res.json({
    method: 'POST',
    a,
    b,
    result: a + b
  });
});

app.listen(PORT, () => {
  console.log(`Server is running: http://localhost:${PORT}`);
});
```

Запуск проекта

Для запуска проекта сначала были установлены зависимости:

```
npm install
```

После этого сервер был запущен командой:

```
npm start
```

После запуска в терминале появилось сообщение:

```
Server is running: http://localhost:3000
```

Это означает, что сервер успешно запущен и доступен по адресу:

```
http://localhost:3000
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Георгий\Desktop\cmp_practicum\Lab05> npm install

added 70 packages, and audited 71 packages in 3m

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Георгий\Desktop\cmp_practicum\Lab05> npm start

> start
> node server.js

Server is running: http://localhost:3000
|
```

Проверка GET-запроса через браузер

Для проверки работы GET-запроса был открыт адрес:

```
http://localhost:3000
```

Сервер вернул JSON-ответ с описанием приложения и доступных маршрутов.

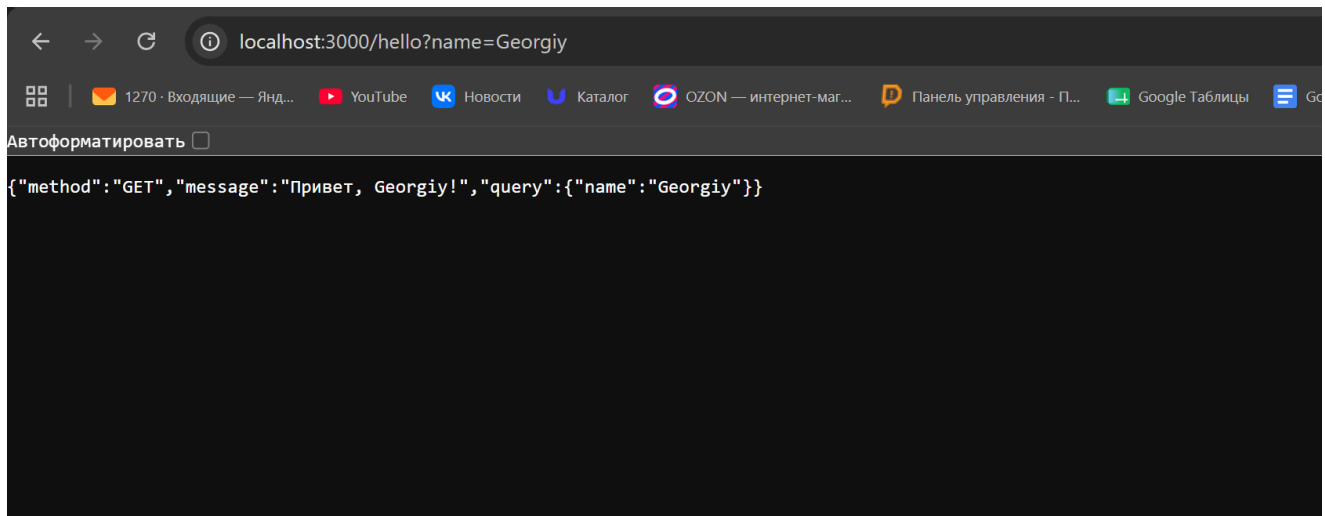
Место для скриншота: результат открытия корневого маршрута в браузере.

Также был проверен маршрут:

```
http://localhost:3000/hello?name=Georgiy
```

В ответ сервер вернул JSON-объект с приветствием:

```
{
  "method": "GET",
  "message": "Привет, Georgiy!",
  "query": {
    "name": "Georgiy"
  }
}
```

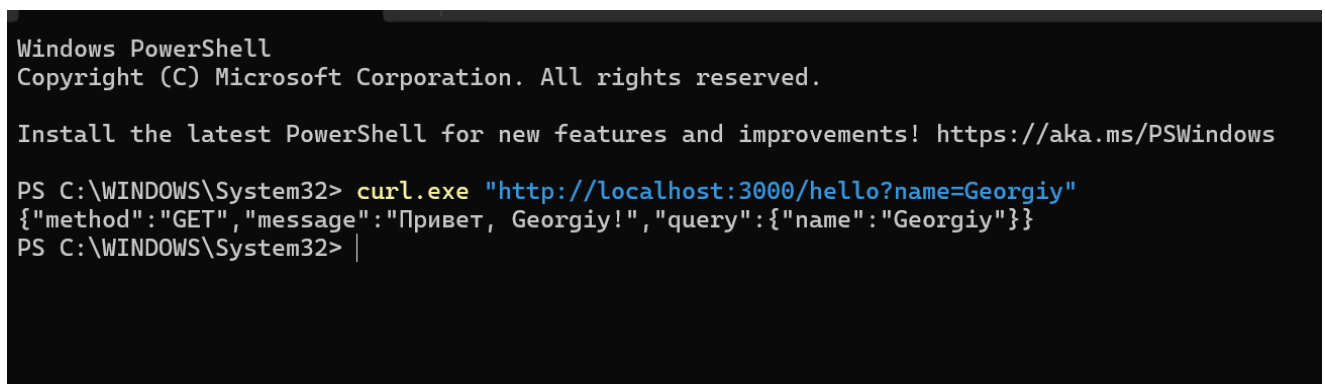


Проверка GET-запроса через cURL

Для отправки GET-запроса из командной строки была использована команда:

```
curl.exe "http://localhost:3000/hello?name=Georgiy"
```

В результате сервер вернул JSON-ответ с переданным именем.



Проверка POST-запроса /echo

Для проверки POST-запроса был использован маршрут /echo .

Команда:

```
curl.exe -X POST "http://localhost:3000/echo" -H "Content-Type: application/json" -d '{"text": "test", "author": "Georgiy"}'
```

Сервер принял JSON-данные и вернул их обратно в ответе:

```
{  
  "method": "POST",
```

```
"message": "Данные успешно получены",
"received": {
  "text": "test",
  "author": "Georgiy"
}
}
```

```
PS C:\WINDOWS\System32> curl.exe -X POST "http://localhost:3000/echo" -H "Content-Type: application/json" -d '{"text":"test","author":"Georgiy"}'
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>SyntaxError: Expected property name or &#39;&#39; in JSON at position 1 (line 1 column 2)<br> &nbsp; &nbsp; &nbsp;at JSON
.parse (&lt;anonymous&gt;)<br> &nbsp; &nbsp; &nbsp;at parse (C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\body-par
ser\lib\types\json.js:92:19)<br> &nbsp; &nbsp; &nbsp;at C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\body-parser\l
ib\read.js:128:18<br> &nbsp; &nbsp; &nbsp;at AsyncResource.runInAsyncScope (node:async_hooks:227:14)<br> &nbsp; &nbsp; &nbsp;at invoke
Callback (C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\raw-body\index.js:238:16)<br> &nbsp; &nbsp; &nbsp;at done (
C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\raw-body\index.js:227:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMessage.o
nEnd (C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\raw-body\index.js:287:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMes
sage.emit (node:events:509:28)<br> &nbsp; &nbsp; &nbsp;at endReadableNT (node:internal/streams/readable:1735:12)<br> &nbsp; &nbsp; &nbsp;at process.processTicksAndRejections (node:internal/process/task_queues:90:21)</pre>
</body>
</html>
PS C:\WINDOWS\System32> |
```

Проверка POST-запроса /sum

Для проверки обработки данных был реализован маршрут /sum , который принимает два числа и возвращает их сумму.

Команда:

```
curl.exe -X POST "http://localhost:3000/sum" -H "Content-Type: application/json" -d '{"a":10,"b":15}'
```

Сервер вернул результат сложения:

```
{
  "method": "POST",
  "a": 10,
  "b": 15,
  "result": 25
}
```

```
PS C:\WINDOWS\System32> curl.exe -X POST "http://localhost:3000/sum" -H "Content-Type: application/json" -d '{"a":10,"b":15}'
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>SyntaxError: Expected property name or &#39;&#39; in JSON at position 1 (line 1 column 2)<br> &nbsp; &nbsp; &nbsp;at JSON
.parse (&lt;anonymous&gt;)<br> &nbsp; &nbsp; &nbsp;at parse (C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\body-par
ser\lib\types\json.js:92:19)<br> &nbsp; &nbsp; &nbsp;at C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\body-parser\l
ib\read.js:128:18<br> &nbsp; &nbsp; &nbsp;at AsyncResource.runInAsyncScope (node:async_hooks:227:14)<br> &nbsp; &nbsp; &nbsp;at invoke
Callback (C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\raw-body\index.js:238:16)<br> &nbsp; &nbsp; &nbsp;at done (
C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\raw-body\index.js:227:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMessage.o
nEnd (C:\Users\Георгий\Desktop\cmp_practicum\Lab05\node_modules\raw-body\index.js:287:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMes
sage.emit (node:events:509:28)<br> &nbsp; &nbsp; &nbsp;at endReadableNT (node:internal/streams/readable:1735:12)<br> &nbsp; &nbsp; &nbsp;at
 process.processTicksAndRejections (node:internal/process/task_queues:90:21)</pre>
</body>
</html>
PS C:\WINDOWS\System32> |
```

Вывод

В ходе выполнения лабораторной работы было разработано веб-приложение, которое обрабатывает GET- и POST-запросы. Были реализованы маршруты для проверки работы сервера, получения данных из query-параметров, обработки JSON-данных и выполнения простого вычисления.

Работа приложения была проверена через браузер и через cURL. Все ответы сервера возвращаются в формате JSON, что соответствует требованиям задания.